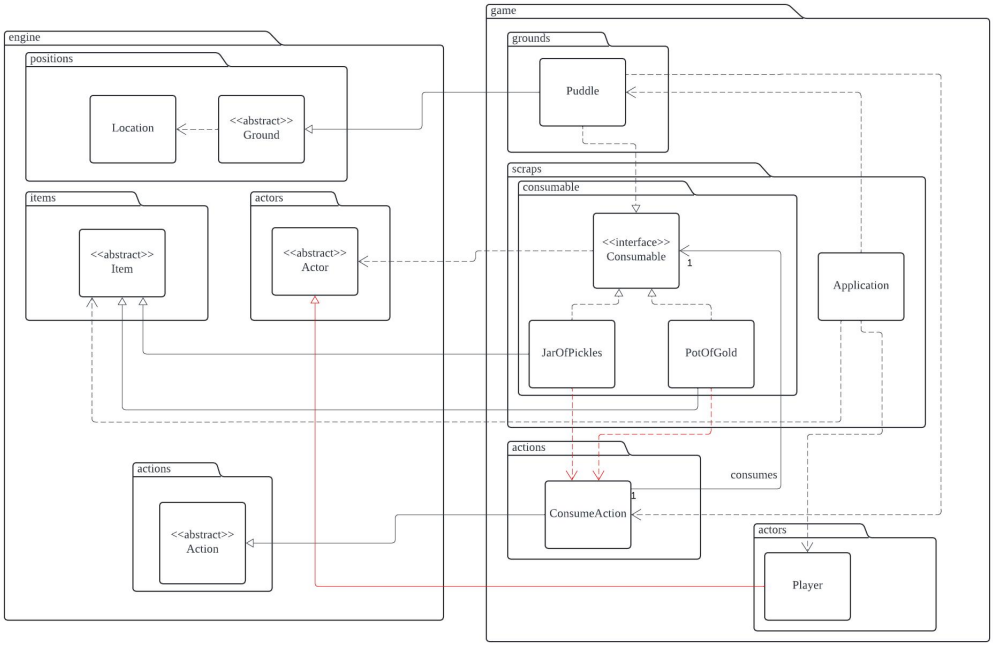
Assignment 2 Design Rationale

Requirement 3: More Scraps

UML Diagram:



Design Goal:

The design goal for this assignment is to introduce new scrap items and interactions for the player to increase the diversity of the gameplay while adhering closely to relevant design principles and best practices. These additions will enrich the gameplay such as providing benefits or consequences to the player.

Design Decision:

In implementing new scrap items such as jar of pickles, pot of gold and a new in-game interaction such as drinking water from the puddle, the decision was made to create concrete classes for the new scrap items which extends from the Item abstract class and implements the Consumable Interface. Then, each of the scrap concrete classes have a ConsumeAction object stored inside the ActionList from the method allowableActions(). Moreover, concrete class Puddle which extends from Ground abstract class also implemented Consumable Interface and has a ConsumeAction object stored inside the allowableActions() ActionList. This is illustrated in the UML diagram. Compared to the design for Assignment 1, the logic of removing the consumable item once the player consumed is shifted from ConsumeAction to the consumedEffect() for each item concrete classes that implemented Consumable Interface.

[Explain the rationale behind the decision, including considerations of efficiency, scalability, maintainability, or other relevant factors]. The rationale behind creating concrete classes for the new scrap items which extends from Item abstract class and Consumable Interface was to ensure a flexible and scalable design since the core functionality of items is inside the Item abstract class while the individual unique attributes of each scrap items are encapsulated inside their own concrete classes. This makes it easier to create new various type of scraps without making an extensive modification to the existing code. It is also friendly for code maintainability since each consumable behaviour and logic is defined within its respective class which reduces the dependencies between different parts of code in different classes. Moreover, the shifting of logic for the removing consumable item from ConsumeAction to the consumedEffect() method also provides better maintainability by centralising the code to manage item consumption inside the scrap item classes themselves. This change is also made to satisfy the requirement of Puddle being a "consumable" while it is not an "item".

Alternative Design:
One alternative approach could involve creating a ConsumableScrap abstract class which extends from Item abstract class and implement the Consumable Interface. However, this is considered an over-abstraction as the ConsumableScrap does not provide a significant benefit for code reuse. Moreover, it adds an extra layer of abstraction and violates the principle of maintaining a simple design. Other than that, each scrap item could have unique attributes beyond to just simply be consumable by player. On the other hand, the original decision is preferred on its simplicity and maintainability.

Analysis of Alternative Design:
The alternative design is not ideal because it violates various Design and SOLID principles:

1. **Single Responsibility Principle:**
    - ConsumableScrap is responsible for maintaining both consumable behaviour and managing item which violates the separate of concerns.
2. **Open-Closed Principle:**
    - The alternative design restricts the potential extensibility of the code. If a ConsumableScrap abstract class is introduced, the design limits other various kind of scrap items with unique behaviour which do not fall under the category of ConsumableScrap. It could be necessary to alter the ConsumableScrap class to accommodate new types of scrap items which goes against the principle of open for expansion but closed for modification concept.

Final Design:

In our design, we closely adhere to the relevant design principles such as SRP, OCP and ISP from SOLID principles which ensure a flexibile and maintainable codebase:

1. **Single Responsibility Principle (SRP):**
    - **Application:** Each item concrete classes such as Jar of Pickles, Pot of Gold in our final design bears a single responsibility with a specific aspect of functionality to scrap items.
    - **Why:** It promotes code maintainability, readability since each item concrete class is focused on a single concern.
    - **Pros/Cons:** It eases the effort of maintaining the entire code as the impact of changes to individual classes are reduced. However, this results in creation of many number of classes.
2. **Open-Closed Principle (OCP):**
    - **Application:** If there is a new scrap item that is consumable such as Burger, our code can be easily extended to implement the item without modifying the existing code
    - **Why:** The design is open for extension but closed for modification which reduces the risk of bugs appearing and promotes code extensibility.
    - **Pros/Cons:** The design maintains potential evolution of the game when many more scrap items or interactions introduced with minimal risk. However, the system is required to be maintained with careful planning as it evolves.
3. **Interface Segregation Principle (ISP):**
    - **Application:** Our design defines a clear interface which represents the specific functionality, which promotes the flexibility. For example, Consumable interface represents the “Consumable” behaviour in the game
    - **Why:** This ensures that classes are relying on interfaces or functionality they do not possess which reduces the coupling.
    - **Pros/Cons:** The design choice promotes code flexibility and it is easy to maintain as classes are only permitted to depend on the interface they required. However, large number of interfaces will be created based on each unique functionality required for the classes

Conclusion:

Overall, our chosen design provides a robust framework for Assignment 2 Requirement 3 which introduces new scrap items such as Jar of Pickles, Pot of Gold and interaction such as drinking water from the puddle. By carefully considering the SOLID Principle, we have developed a solution that is scalable and easy to maintain paving the way for future extensions such as introduction of new scrap items and new Action with the scrap items.